VayuNetra

Frontend Handover & Integrator Guide

Purpose: A practical handover document describing the frontend work completed, current demo limitations, the expected integrated behavior, and the responsibilities of the integration team.

Handover status: Frontend demonstration substantially implemented. Perception simulation is intentionally not marked final because its behavior was not reliable enough to treat as production-ready.

1. What This Frontend Is

The VayuNetra frontend is the operational presentation/control layer. It provides Dashboard, Missions, Drones, Live Map, Perception and History pages. During development, demo data and browser localStorage are used so the interface can demonstrate the intended workflow without the final backend, perception pipeline, localisation system, swarm controller or hardware telemetry.

The frontend must not be the source of truth for real drone state. In the integrated system, real services should supply mission state, drone state, positions, sensor information, perception detections and operation history.

2. Work Completed

Area	Completed work
Navigation	Dashboard, Missions, Drones, Live Map and Perception routes are present. A History route/page was added.
Mission lifecycle	Mission start, persistent active operation, stop confirmation, moving stopped/completed operations to History, and unique
Operation persistence	operationStorage.js provides active-operation and history functions using localStorage for the demo.
Fleet	A 128-drone demonstration fleet was created. Intended station baseline: 100 AVAILABLE, 12 ACTIVE, 8 CHARGING and
Drone records	Demo drone records contain ID, latitude, longitude, status, battery, mission and survivor count.
Live Map	Operational map is intended to show only drones assigned to the current operation rather than all station drones.
Fullscreen map	Fullscreen mode is intended to hide the surrounding drone section and let the map fill the available laptop/browser viewpo
History	History is presented in table format with operation information, date/time, status and delete capability.
UI preservation	Existing VayuNetra visual configuration was retained as the preferred baseline.

3. Current File Responsibilities

File/page	Responsibility now	Integration target
App.jsx	Global layout, navigation and routes	Keep routes; connect pages to real services/state
Dashboard.jsx	Overall operational summary	Live KPIs, alerts, mission and fleet state
Missions.jsx	Mission start/active/stop UI	Mission controller/orchestrator commands and real status
Drones.jsx	Fleet list and status display	Real drone registry + telemetry
LiveMap.jsx	Operational map and drone view	Real localisation + assigned-drone positions/routes
Perception.jsx	Demo images/detection display	Real RGB/thermal/YOLO outputs
History.jsx	Historical operation table	Backend/database history
demoData.js	Mock drone/disaster/detection data	Replace with live data adapters
operationStorage.js	Browser active/history storage	Backend persistence or synchronized store

4. One Source of Truth

The biggest integration risk is allowing each page to calculate its own version of reality. If an operation assigns DR-003 and DR-007, Missions, Drones and Live Map must all obtain that same assignment. Live Map must never simply render every drone whose status happens to be ACTIVE in demoData.js.

Recommended operation state:

```
{ "id": "OP-001", "name": "Flood Rescue", "status": "ACTIVE", "disasterType": "Flood", "assignedDrones":
["DR-003", "DR-007"], "progress": 35, "startTime": "2026-08-28T20:00:00Z" }
```

The production schema may differ, but it must provide a unique operation ID, lifecycle status, mission/disaster type, assigned drone IDs, progress and timestamps.

5. Mission Lifecycle

- **Start:** Create a unique operation ID, allocate eligible drones, persist the operation as ACTIVE, and return the assigned-drone list.
- **During operation:** Update mission progress, drone telemetry, localisation and perception results. The operation remains active while navigating between pages.
- **Refresh/reopen:** Fetch the active operation from the authoritative service; do not require the user to simulate again.
- **Stop:** Frontend requests confirmation, then sends a stop/cancel command. The controller stops the operation and releases/updates assigned drones.
- **Complete:** At 100%, mark the operation COMPLETED and move it to History.
- **History:** Store the final record with operation ID, details, assigned drones, start/end timestamps and final status.

6. Drone Allocation & Fleet

The current demo contains 128 station drones. Station fleet and operational fleet are different concepts. All 128 can exist in the registry, but only drones assigned to the current operation should appear as mission markers on Live Map.

- Station fleet: all registered drones.
- Available: eligible for allocation.
- Active: currently assigned to active operations.
- Charging: temporarily unavailable.
- Unavailable: offline/maintenance/ineligible.
- Allocated drones: exact IDs returned by the mission allocator.

Critical: Do not hard-code different active counts on different pages. Counts should be derived from the same live fleet/operation state.

7. Live Map Integration

- Localisation supplies latitude, longitude, heading and timestamp for each drone.
- Live Map filters the fleet by the current operation's assignedDrones.
- Only those assigned IDs are rendered as mission markers.
- Station-only drones remain visible in the fleet page but not as active mission markers.
- Routes/paths come from mission planning/localisation, not fake frontend movement.
- Fullscreen resizes the map to the actual viewport and hides the surrounding drone section.

8. Perception — Not Final

Perception is explicitly not marked complete. The current frontend has five demonstration images and detection presentation, but the attempted simulation/filter behavior was not reliable enough to hand over as a finished implementation.

Required integrated behavior:

- RGB and/or thermal frames are received from the drone sensors.
- The perception model determines disaster/object classes and confidence.
- The selected/active disaster scenario must not be mixed with unrelated images or detections.
- Image, detection class, confidence, sensor and drone ID must belong to the same event.
- Every detection should carry timestamp and, where applicable, geographic position.
- Perception results must be associated with the drone that produced the frame.
- The frontend displays model output; it should not randomly invent production detections.

Suggested perception event:

```
{ "operationId": "OP-001", "droneId": "DR-003", "timestamp": "2026-08-28T20:12:31Z", "disasterType":
"Earthquake", "object": "Structural Damage", "confidence": 0.86, "sensor": "RGB", "imageUrl": "...",
"latitude": 15.3068, "longitude": 75.7028 }
```

9. RGB / Thermal / YOLO Integration

- **RGB:** camera frame → preprocessing → model → detections → timestamp/drone/location metadata.
- **Thermal:** thermal frame → preprocessing → detection/classification → metadata.
- **Fusion, if used:** associate RGB and thermal detections using timestamp, drone ID and spatial correspondence.
- **YOLO:** expose class, confidence and bounding box through a stable frontend-friendly schema.
- **Frontend:** render the result and image; do not determine disaster type by random image selection.

10. Backend/API Expectations

Purpose	Conceptual interface	Output
Active operation	GET active operation	Current operation + assigned drone IDs
Start mission	POST start operation	New operation ID + allocation
Stop mission	POST stop operation	Confirmed transition/final state
Fleet	GET/stream drones	ID, status, battery, position
Telemetry	WebSocket/stream	Live position, battery, health
Perception	WebSocket/stream	Detection events + image/frame reference
History	GET operation history	Completed/stopped operations
Delete history	DELETE history record	Deletion confirmation

11. Replace These Demo Behaviors

- Hard-coded fleet counts once live telemetry exists.
- Random perception selection as a substitute for model output.
- Static survivor/detection counts once perception is connected.
- Static coordinates once localisation is connected.
- localStorage as the authoritative operational database.
- Frontend-generated mission progress once the controller reports real progress.

12. Do Not Change Without a Requirement

- Overall VayuNetra navigation structure.
- Existing visual configuration that was accepted as the preferred/older page configuration.
- Operation lifecycle: active → stop/complete → history.
- Unique operation ID concept.
- Distinction between station fleet and mission-assigned drones.
- Live Map rule: operational markers represent assigned mission drones, not every registered station drone.

13. End-to-End Acceptance Test

#	Test	Expected result
1	Start mission	Unique operation created; drones allocated
2	Open Drones	Same allocated drones show assignment
3	Open Live Map	Only allocated mission drones visible
4	Open Perception	Results belong to active operation/drone
5	Refresh	Same active operation remains
6	Return to Missions	Mission remains ACTIVE with correct progress
7	Stop mission	Confirmation is required
8	Confirm stop	Operation stops; assigned drones released/updated
9	Open History	Same ID appears with date/time and STOPPED status
10	Delete history	Record disappears
11	Complete another mission	At 100%, operation becomes COMPLETED in History
12	Fullscreen Live Map	Map fills viewport; drone panel hidden
13	Fleet check	128 station drones remain without polluting active map markers

14. Recommended Integration Order

1. Define the authoritative operation schema/API.
2. Connect active operation state to Missions and persistence.
3. Connect the real drone registry and telemetry.
4. Connect mission allocation so assignedDrones is authoritative.
5. Connect localisation to Live Map and verify marker filtering.
6. Connect mission progress, stop and completion events.
7. Connect backend history storage.

8. Connect RGB/thermal/YOLO perception events.
9. Finalize Perception only after real model outputs are available.
10. Run the complete acceptance test.

15. Final Handover Note

The frontend establishes the operational UI and intended workflow. The integration team must now make every page reflect the same real system state: one operation ID, one assigned-drone list, one source of truth for drone status, real localisation for map positions, real perception events for detections, and persistent backend history.

Known limitation: Perception simulation/filtering is not final. The current demo must not be represented as the real perception pipeline. The integrator must connect the actual disaster classification/object-detection pipeline and enforce operation/disaster/image consistency.

Document prepared as a frontend-to-integration handover guide.